Lecture # 23

Date: 18 NOV 25 Day: Tuesday

Strongly connected components

→ Apply DFS
→ Maintained Stack
→ Transpose of the orig graph
→ Apply DFS on G^T.

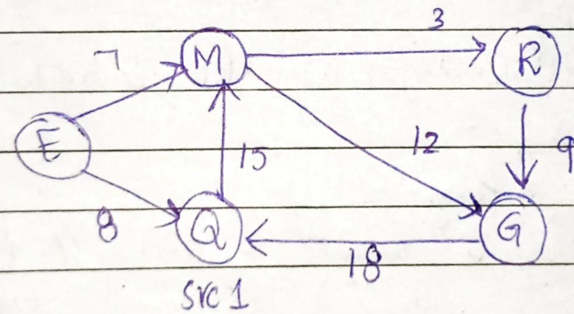
$(T(N) = (V+E) + (V+E) + (V+E)$
$= 3(V+E)$
$= O(V+E)$
$E \approx V^2$

SCC(G){
Initiaze vertices of G

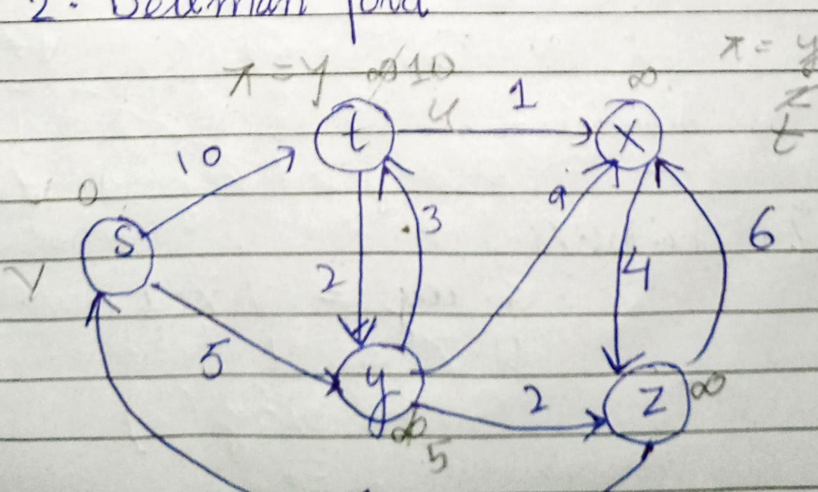
Shortest Path Algorithms

Single source shortest path

pick one vertex & compute shortest path.



1. Dijkstra (in some cases fails on -ive weights)
2. Bellman ford



min heap

Date:

Day:

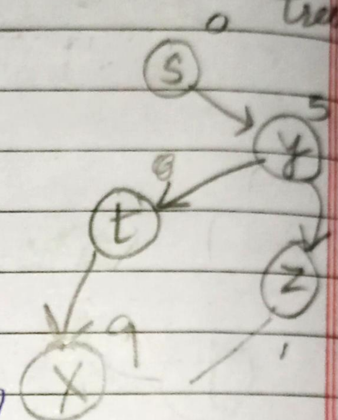
~~s~~	z	t	x	y
0	∞	~~∞~~ 10	∞	~~∞~~ 5

~~y~~	t	z	x
5	~~10~~ 8	~~∞~~ 7	~~∞~~ 14

~~z~~	t	x
7	8	~~14~~ 13

~~t~~	x
	~~13~~ 9

Shortest path tree



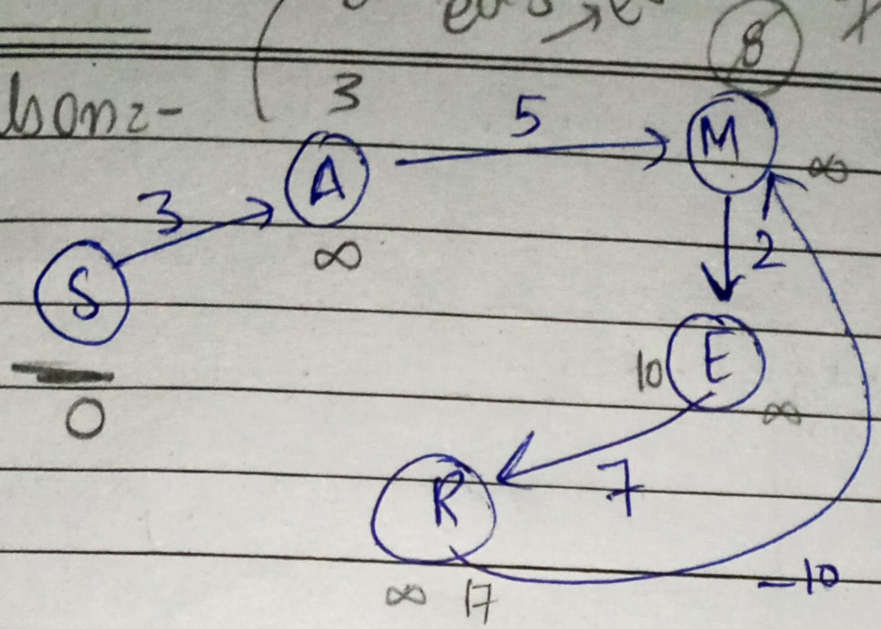
* $\boxed{T(N) = V^2 \log V}$

1. Initialization of all vertices. ($key == \infty$) ↳ V
2. src.key = 0
3. Create a min heap & insert all vertices $V * \log V$

```
while (Q != { }) {
    v . Q.ExtMin()                  V*logV
    for each u ε G.Adj[v]
        if (u.key > v.key + w(v,u))
                                    0 + 10 w(v,u)
if('v' belongs to Q && ...) ∞
            u.key = v.key + w(v,v);
            u.π = u     ↳ predecessor
                          cost      cost of vertex
            Q.Heapify()
              E*logV
```

in the case of -ive edges of cycle
→ Everything sucks 6 7 loop

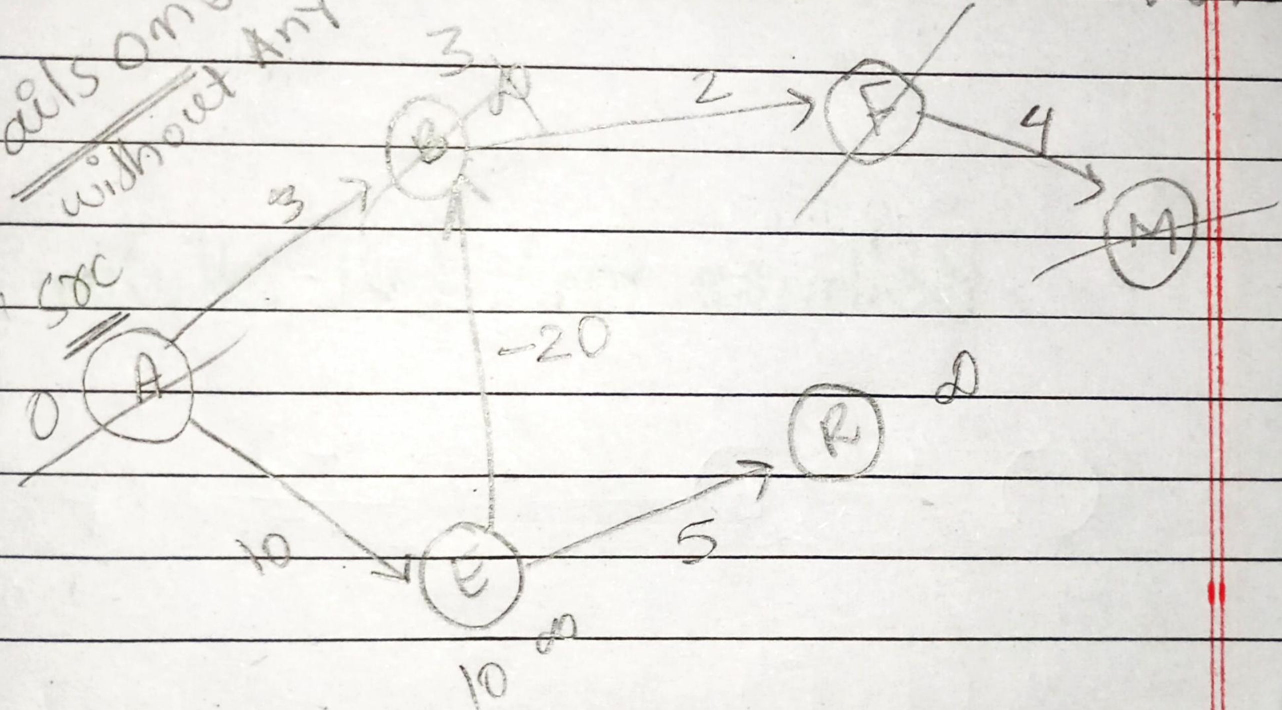
Fails on:-



→ fails on 8 process hojany vertex ki cost kisi negative weight sy improve ho rhi ho.

S	A	M	E	R
0				17

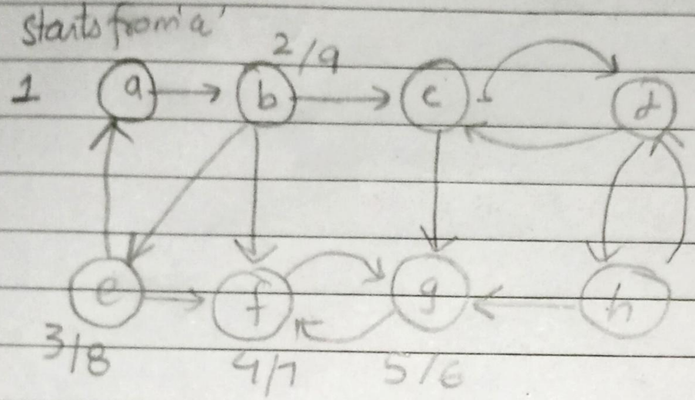
Fails on without Any cycle:



Lecture# 24

Date: Nov 20.25 Day: Thursday

if we use queue instead of stack:

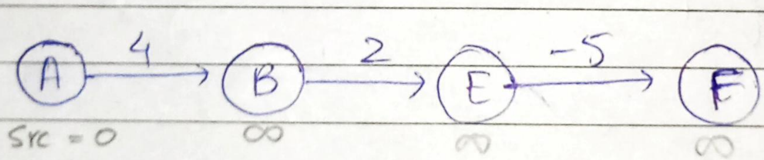


Queue | g | f | e | → does belong to src comp.

we need this order
sink, other, source component

edge based approach:

Bellman Fond (DP-solution)



$w(E, F) = -5$
$w(B, E) = 2$
$w(A, B) = 4$

```
for i = 1 to |G.V| -1
    for each edge(u, v) ε G.E
Relax{  if (v.cost > u.cost + w(u, v))
            v.cost = u.cost + w(u, v)
for each edge (u, v) ε G.V
    if v.d > v.d + w(u, v)
        return false  → negetive cycle
return true
```

Relax (u, v, w):

if v.d > u.cost + w(u, v)

v.cost = u.cost + w(u, v)

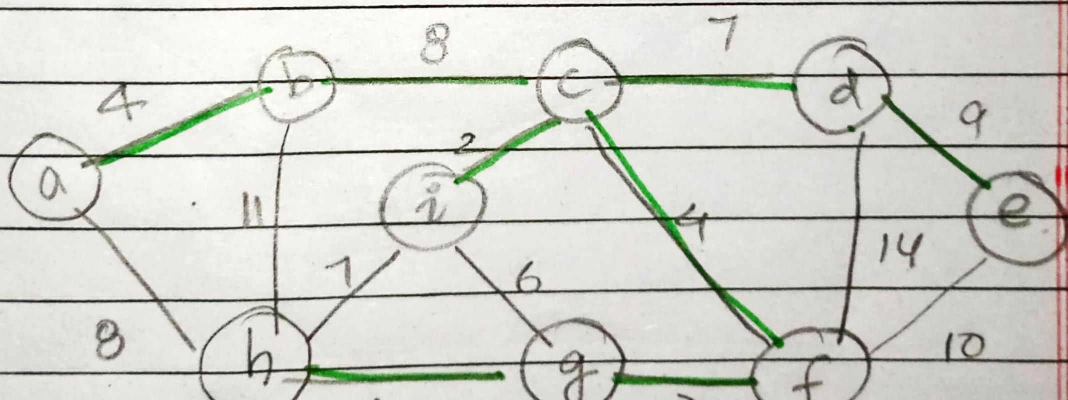
$$T(N) = E * (V-1)$$
$$= E * V$$
$$= V^2 * V$$
$$= V^3$$

(But return actual minimum cost)

network cost → minimum

→ covers all vertices of graph

Minimum Spanning Tree → no cycle



28 30 34 3

4 + 8 + 7 + 9 + 2 + 4 + 2 + 1

= 37